NVIDIA GPU 与 GPGPU 处理器硬件架构综述

从 SIMT 执行模型到 AI 时代的架构演进

技术调研报告　Claude (AI Assistant)

2026 年 4 月

摘要

随着人工智能（尤其是大语言模型）对算力需求的指数级增长，GPU 已从早期的图形渲染加速器演变为现代高性能计算与 AI 训练推理的核心引擎。NVIDIA 凭借其 CUDA 生态与持续迭代的架构创新，确立了在通用 GPU（GPGPU）领域的主导地位。本综述系统梳理了 GPGPU 的硬件设计原理与 NVIDIA GPU 的架构演进脉络，重点分析从 Volta 到 Blackwell 各代 Tensor Core 的技术革新、流式多处理器（SM）的微架构细节（包括 warp 调度、控制流管理、编译器驱动的依赖追踪机制）、内存层次结构的优化策略（TMA、异步拷贝、寄存器堆设计），以及多 GPU 互连体系。此外，本文还将 NVIDIA 的 GPU 设计与 Groq LPU、SambaNova SN40L RDU 等非 GPU 加速器进行对比，探讨领域专用架构（DSA）与通用可编程架构之间的设计权衡。最后，本文展望了面向未来 LLM 负载的 GPU 架构发展趋势与开放研究问题。

关键词： GPGPU、SIMT、Tensor Core、NVIDIA Hopper、Warp Scheduler、内存墙、数据流加速器

目录

1 引言 **4**

1.1 研究背景 4

1.2 综述范围与结构 4

2 GPGPU 基础架构与编程模型 **5**

2.1 从 SIMD 到 SIMT：并行执行模型的演进 5

2.2 CUDA 执行模型：Grid → Block → Warp → Thread 5

2.3 内存层次结构 6

2.4 Kernel 启动与 Host-Device 交互 7

3 流式多处理器（SM）微架构深度剖析 **8**

3.1 SM 的宏观结构与资源分配 8

3.2 Warp Scheduler 与指令流水线 8

3.2.1 ”One Loop” 与 ”Two Loop” 近似模型 8

3.2.2 调度策略：GTO、LRR 与双发射 9

3.3 依赖追踪：从 Scoreboard 到编译器控制位 9

3.3.1 Scoreboard 机制 9

3.3.2 编译器控制位（Control Bits）：NV 的软硬件协同设计 10

3.4 分支处理与控制流演进 10

3.4.1 Pre-Volta：SIMT Stack 与死锁问题 10

3.4.2 Post-Volta：显式同步指令与 Hanoi 微架构 11

3.5 寄存器文件设计与 Operand Collector 11

3.5.1 寄存器文件面临的挑战 11

3.5.2 Register File Banking 与 Operand Collector 11

3.5.3 寄存器层次结构与复用优化 12

4 Tensor Core 架构演进与软硬件协同 **12**

4.1 从 CUDA Core 到 Tensor Core：动机与背景 12

4.2 Volta：第一代 Tensor Core（Warp-scoped MMA） 13

4.3 Turing：低精度扩展与第二代 Tensor Core 13

4.4 Ampere：异步拷贝、稀疏性与第三代 Tensor Core 13

4.4.1 第三代 Tensor Core 核心能力 13

4.4.2 异步数据拷贝（cp.async） 14

4.5 Hopper：WGMMA、TMA 与第四代 Tensor Core 14

4.5.1 Warpgroup-level MMA（WGMMA） 14

4.5.2 Tensor Memory Accelerator（TMA） 14

4.5.3 FP8 与 Transformer Engine 15

4.6 Blackwell：第五代 Tensor Core 与 TMEM 架构分析 15

4.6.1 Tensor Memory（TMEM） 15
4.6.2 CTA Pair 与双 SM 协作 15
4.6.3 精度进一步降低：MXFP 与 NVFP4 16
4.6.4 对 Blackwell 架构的批判性分析 16
4.7 Tensor Core 演进趋势总结 16

5 CuTe Tile 编程与 Layout 代数 17
5.1 CuTe 的架构定位与编程动机 17
5.2 Layout 代数基础：Shape、Stride 与 Tensor 17
5.3 Tile 编程模型：Global → Shared → Register 18
5.4 MMA_Atom、TiledMMA 与 ThrMMA 的抽象层次 19
5.5 CuTe 的架构意义 20

6 内存层次与数据移动优化 21
6.1 GPU 内存层次结构全景 21
6.2 寄存器文件设计：不仅仅是 SRAM 21
6.3 共享内存与 Bank Conflict 21
6.4 L1/L2 缓存与 Cache Line 管理 22
6.5 异步数据搬运：cp.async 与 TMA 22
6.6 多 GPU 互连与内存一致性 22

7 NVIDIA GPU 架构代际对比 23
7.1 从 Pascal 到 Blackwell：计算能力演进 23
7.2 SM 微架构的代际差异 23
7.3 精度演进与 AI 训练的匹配 24

8 替代加速器架构对比 24
8.1 Groq LPU：确定性执行与张量流式处理器 24
8.2 SambaNova SN40L RDU：可重构数据流单元 25
8.3 设计哲学差异总结 26

9 研究趋势与未来挑战 26
9.1 面向 LLM 的架构瓶颈 26
9.2 稀疏化、量化与近存计算 26
9.3 领域专用 vs 通用可编程的权衡 27
9.4 编译器-硬件协同设计的深化 27

10 结论 27

1 引言

1.1 研究背景

自 2012 年 AlexNet 在 ImageNet 竞赛中利用 NVIDIA GPU 取得突破性成果以来，GPU 在通用并行计算（General-Purpose computing on GPU, GPGPU）领域的地位迅速攀升。随后，深度学习模型的参数量与训练数据规模呈现指数级增长：从 BERT（3.4 亿参数）到 GPT-3（1750 亿参数），再到如今的 GPT-4 级别模型（据传超过 1.8 万亿参数），对计算硬件提出了前所未有的挑战。GPU 凭借其大规模的线程级并行（Thread-Level Parallelism, TLP）和高内存带宽，成为训练与部署这些模型的首选平台。

NVIDIA 作为 GPU 市场的领导者，通过持续不断的架构迭代 —— 从 2016 年的 Pascal，到 2017 年的 Volta、2018 年的 Turing、2020 年的 Ampere、2022 年的 Hopper，再到 2024 年的 Blackwell —— 在单芯片算力、内存带宽、互联能力、专用 AI 加速单元（Tensor Core）等方面实现了显著跃升。例如，H100 相比 A100 在 FP8 精度下可实现高达 6 倍的性能提升，而 Blackwell 进一步引入了第五代 Tensor Core 和新的 Tensor Memory（TMEM）机制。

然而，GPU 架构的复杂性也在同步增长。理解其硬件设计不仅需要考虑传统的处理器微架构问题（指令调度、寄存器分配、缓存一致性），还必须深入分析 SIMT 执行模型、warp 级同步与分支发散、编译器与硬件的协同设计（如 control bits）、以及为矩阵运算专门优化的 Tensor Core 流水线。这些知识点分散在学术论文、技术博客、专利文档和官方白皮书之中，缺乏一份系统性的中文综述将其整合。

1.2 综述范围与结构

本综述的核心定位是**从硬件微架构视角理解 NVIDIA GPU 与 GPGPU 的设计原理**。具体来说：

第二章介绍 GPGPU 的编程模型（CUDA/SIMT）、执行层次结构（Grid/Block/Warp/Thread）和内存模型，为后续微架构分析奠定概念基础。第三章深入剖析流式多处理器（SM）的内部结构，包括 warp scheduler、分支处理、scoreboard/operand collector、以及控制流设计的演进（SIMT-stack 到显式同步指令）。第四章专门讨论 Tensor Core 的架构演进与软硬件协同，覆盖从 Volta 的 WMMA 到 Blackwell 的 UMMA 的完整技术栈。

第五章分析 GPU 的内存层次结构（寄存器堆、共享内存、L1/L2/HBM）、异步数据传输机制（TMA/cp.async/Mbarrier），以及多 GPU 互联技术（NVLink/NVSwitch）。第六章横向对比 Pascal 到 Blackwell 各代架构的关键参数与设计哲学演变。第七章将 NVIDIA GPU 与 Groq LPU、SambaNova SN40L RDU 进行对比，分析通用 GPU 与数据流加速器在 AI 工作负载上的差异。第八章讨论 LLM 时代 GPU 面临的核心瓶颈（内存墙、多卡扩展、稀疏性/量化支持），并提出未来研究方向。

需要说明的是，GPU 生态包含硬件架构、编程模型（CUDA/OpenCL）、编译器与运行时（NVCC、CUTLASS、Triton）、系统软件（NCCL、Driver）等多个层次。本综述聚焦于**硬件微架构**，仅在必要时涉及 PTX 指令、CUDA C++或 CUTLASS 以说明软硬件

接口。此外，由于部分资料来源于技术博客和专利分析，文中涉及 Blackwell 和 Rubin 架构的内容多为基于公开信息的推测性分析，对于这些内容本文会明确标注其推测性质。

2 GPGPU 基础架构与编程模型

2.1 从 SIMD 到 SIMT：并行执行模型的演进

GPU 之所以能够在大规模数据并行任务中展现出远超 CPU 的峰值性能，根本原因在于其采用了与 CPU 截然不同的执行模型。在 CPU 领域，多核处理器（MIMD）的每个核心各自独立完成一个任务，并行效率受限于应用本身的线程级并行度；而 SIMD（单指令多数据流）向量处理器虽然能同时处理多个数据元素，但要求数据在内存中严格连续，否则就需要昂贵的 gather/scatter 操作，且分支发散（divergence）会严重杀伤性能。

GPU 采用的核心执行模型是 **SIMT（Single-Instruction, Multiple-Threads）**。从程序员的视角来看，GPU 编程写的是单个 thread 的逻辑；但从硬件的视角来看，GPU 会将多个 thread 放到一起以 lockstep 方式执行，这一组 thread 称为一个 **warp**。在传统的 Pre-Volta 架构中，一个时钟周期内一个 warp 里的所有线程必须执行完全相同的指令。SIMT 相比 CPU 的 SMT（Simultaneous Multithreading）最大的区别在于：**SIMT 将 thread 暴露给了程序员**，程序员可以通过 `threadIdx` 和 `blockIdx` 等内置变量显式控制每个线程的行为，从而以接近单线程编程的思维方式来编写并行代码。

SIMT 的执行效率高度依赖于数据并行性。当所有线程执行相同路径时，SIMT 的效率接近理想状态；但当遇到 `if` 分支导致部分线程走不同路径时，就会发生**线程发散（divergence）**。在传统设计中，warp 会串行化执行各个分支路径，inactive 的线程被屏蔽掉，这直接造成了计算资源的浪费。关于控制流发散的硬件处理机制以及 Volta 之后独立线程调度（Independent Thread Scheduling）的演进，将在第三章中详细讨论。

2.2 CUDA 执行模型：Grid → Block → Warp → Thread

CUDA 编程模型的核心抽象是一个四级层次结构：**Grid（网格）→ Block（线程块）→ Warp（线程束）→ Thread（线程）**。一个 kernel 函数启动时会配置为一个 Grid，Grid 由多个 Block 组成，每个 Block 又由多个 Thread 组成。现代 NVIDIA GPU 中，一个 warp 固定包含 32 个线程。

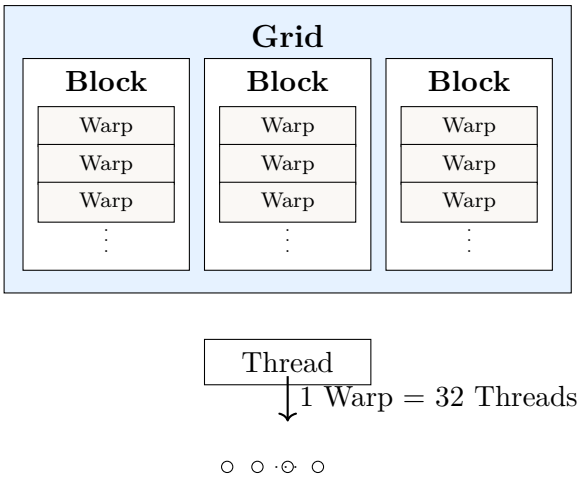

图 1: CUDA 执行模型的四级层次结构。Kernel 启动时配置为一个 Grid，Grid 由多个 Block 组成，每个 Block 包含若干 Warp，每个 Warp 由 32 个 Thread 组成。

如图 1 所示，`threadID` 和 `blockID` 被设计为三维向量，这是因为很多并行计算任务天然具有二维或三维的空间结构。例如，处理一张图像时，每个像素都有 (x, y) 坐标，启动一个二维的线程网格可以让线程 (x, y) 自然地负责处理像素 (x, y)。需要注意的是，**三维 ID 只是给程序员提供的抽象模型**，在硬件层面 GPU 只有一串需要执行的线程列表。

GPU 的调度系统以 **thread block** 为粒度分配任务。一旦一个 thread block 被分配给某个 SM（Streaming Multiprocessor），就会在该 SM 上执行到结束，不会中途迁移到其他 SM。一个 SM 可以同时处理多个 block，具体数量受限于 SM 内的资源（寄存器、共享内存、warp slot 等）。例如，Maxwell 架构的每个 SM 最多可容纳 2048 个线程（即 64 个 warp）。调度器会将 block 分发到空闲的 SM 上，但程序员无法预测某个特定 block 会被分配到哪个 SM，因此所有 block 必须是相互独立的。

在硬件层面，**thread 是一个逻辑概念，warp 才是物理执行单元**。一个 warp 内的 32 个线程共享同一个程序计数器（PC），在同一周期执行同一条指令。每个 thread 虽然被抽象为拥有独立的 PC 和寄存器文件，但硬件上这些寄存器实际上是 **per-warp 的向量寄存器**：硬件将一个 warp 的所有寄存器组织在一起，以向量方式处理。

2.3 内存层次结构

GPU 的内存系统是一个复杂的多层层次结构，从程序员视角可以划分为以下几级：

首先是**寄存器（Register）**，每个 thread 私有的最快存储。从硬件视角看，寄存器实际上是 per-warp 的向量寄存器。GPU 的一个 SM 拥有巨大的寄存器文件（如 Ampere/Hopper 的 256KB），以便所有常驻 warp 的上下文能够常驻片上，实现零开销的 warp 切换。

其次是**共享内存（Shared Memory）**，位于 SM 内部，同一个 block 内的所有 thread 都可以访问。共享内存的延迟远低于全局内存，常被用于线程间的高速数据交换和协作计算。但共享内存存在 **bank conflict** 问题：如果多个线程在同一周期访问同一个 bank，访问会被串行化，从而降低有效带宽。

L1 缓存/共享内存统一子系统从 Fermi 架构开始，NVIDIA 将 L1 数据缓存和共享

内存集成到统一的存储子系统中，可以根据需求灵活划分比例。**L2 缓存**跨 SM 共享的片上缓存，例如 H100 配备了 50MB 的 L2 缓存，用于平滑不同 SM 之间的数据共享和减少 HBM 访问。**全局内存（Global Memory / HBM）**是 GPU 上的主要片外存储，通常采用高带宽显存（HBM），H100 的 HBM3 显存带宽高达 3TB/s，这是 GPU 在处理大规模矩阵运算时的关键优势。最后是**常量内存（Constant Memory）**，只读存储，带有专门的缓存层次。

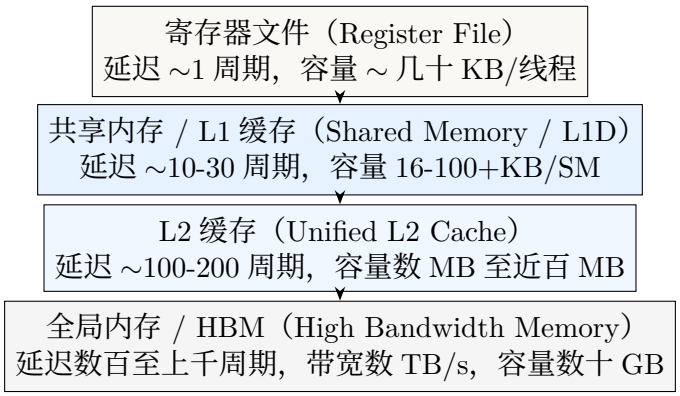

图 2: GPU 内存层次结构示意图。从上到下，容量增大、延迟增加，而带宽在片上层次最高。

在各层级之间搬运数据的开销差异巨大。寄存器访问最快，共享内存次之，而全局内存的延迟可能高达数百个时钟周期。因此，高效 GPU 程序的核心优化目标就是：**最大化数据在快速层级（寄存器、共享内存）中的复用，减少对全局内存的访问**。正如相关研究指出的，”Data Movement More, Memory Reference More, Energy More” —— 数据搬运是 GPU 能耗的主要来源。

2.4 Kernel 启动与 Host-Device 交互

典型的 CUDA 程序执行流程如下：CPU（Host）分配设备内存（`cudaMalloc`）；将输入数据从主机内存拷贝到设备全局内存（`cudaMemcpyHostToDevice`）；启动 kernel（`kernel<<<DimGrid, DimBlock>>>(args)`），在 GPU（Device）上并行执行；计算完成后，将结果从设备拷贝回主机（`cudaMemcpyDeviceToHost`）；最后释放设备内存（`cudaFree`）。

Kernel 内部的每个线程通过全局索引计算自己应该处理的数据位置：

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

需要注意的是，当数据总量 n 不是 block size 的整数倍时，末尾的 block 会有一部分线程” 越界”。这时需要通过 `if (i < n)` 进行保护。但这种保护本身就会导致**线程发散**：同一个 warp 中部分线程执行计算，部分线程跳过，造成了一定的资源浪费。这是 GPU 编程中常见的边界处理开销。

3 流式多处理器（SM）微架构深度剖析

3.1 SM 的宏观结构与资源分配

流式多处理器（Streaming Multiprocessor, SM）是 NVIDIA GPU 的核心计算单元。一个 SM 可以被视为一个功能完备的小型处理器，内部包含了完整的取指、译码、发射、执行、访存和写回流水线。现代 GPU 拥有数十到上百个 SM（如 H100 有 132 个 SM），通过并行扩展实现海量算力。

SM 内部的两大核心资源是**寄存器文件（Register File）**和**共享内存（Shared Memory）**。这两者在 block 被分配到 SM 时进行静态划分：每个 block 按需占用一定量的寄存器和共享内存，SM 据此决定能同时容纳多少个 block。如果一个 block 占用了过多的寄存器，SM 能同时调度的 block 数量就会减少，从而可能降低整体的延迟隐藏能力。

一个 SM 内部的执行单元包括：**CUDA Core（FP32/INT32 ALU）**负责标量浮点和整数运算；**Tensor Core** 专门加速矩阵乘加运算的硬件单元；**加载/存储单元（LD/ST Unit）**处理内存访问请求；**特殊功能单元（SFU）**处理超越函数（如 `sin`、`exp`、`rcp`）等；以及**纹理单元**处理图形相关的纹理采样请求。

在 Turing 及之后的架构中，Tensor Core 逐渐成为 AI 负载的核心算力来源，而 CUDA Core 则负责非矩阵类的通用计算。值得注意的是，**Ampere 和 Hopper 架构支持 CUDA Core 与 Tensor Core 的并行执行**：当 Tensor Core 在进行矩阵计算时，CUDA Core 可以继续处理其他标量任务，这显著提升了 SM 内多功能单元的利用率。

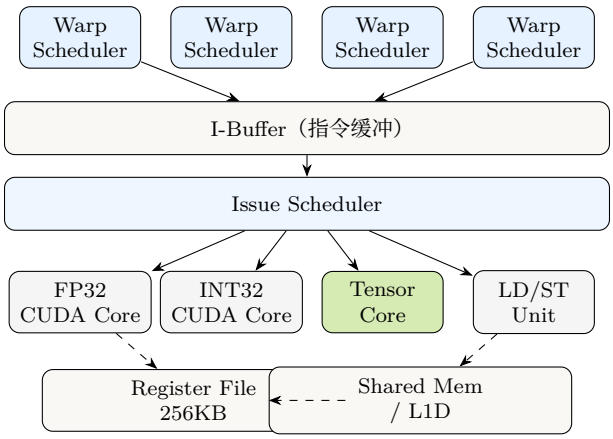

图 3: SM 内部结构简化示意图。包含 4 个 Warp Scheduler、I-Buffer、Issue Scheduler、多种执行单元（CUDA Core、Tensor Core、LD/ST），以及寄存器文件和共享内存/L1。

3.2 Warp Scheduler 与指令流水线

3.2.1 "One Loop" 与 "Two Loop" 近似模型

理解 GPU 的 warp 调度，最简单的方式是从"One Loop" 近似模型开始：每个周期 scheduler 选择一个 warp 取指令，下一个周期切换到不同的 warp，一个 warp 在上一条指令执行完之前不会被再次发射。这个模型极大地简化了 warp 内部的数据依赖追踪 —— warp 严格按顺序执行完一条再取下一条。

但真实 GPU 的调度远比这复杂。现代 GPU 采用**"Two Loop" 模型**，包含两个解耦的调度器：**Fetch Scheduler（Scheduler 1）**负责从不同 warp 中取指令并译码，填充到 **I-Buffer（指令缓冲）**中。如果发生 I-cache miss，硬件不会等待，而是直接放弃本次请求，立即服务其他 warp。**Issue Scheduler（Scheduler 2）**负责从 I-Buffer 中选择就绪的指令发射到执行单元。由于 I-Buffer 中可能同时存在来自几十个 warp 的待执行指令，发射调度器需要在每个周期内快速决定哪些指令可以安全发射。

这两个调度器的解耦是 GPU 实现高效延迟隐藏的关键：即使某个 warp 的指令因为数据没准备好而无法发射，取指调度器也可以继续为其他 warp 取指，不会让流水线停顿下来。

3.2.2 调度策略：GTO、LRR 与双发射

常见的 warp 调度策略包括：**GTO（Greedy-Then-Oldest）**，只要存在就绪指令和空闲执行单元，就立即发射（Greedy）；当有多条指令同时满足条件时，选择等待时间最长（Oldest）的 warp 的指令，以防止饿死。**Loose Round-Robin（LRR）**按顺序轮流服务各个 warp，跳过没有就绪指令的 warp。此外还有 **Two-Level 调度**，第一级选出候选 warp，第二级从中最终确定要发射的指令。

现代 GPU 通常配备多个 warp scheduler（如每个 SM 有 4 个），每个 scheduler 管理一部分 warp，可以在同一个周期内从不同 warp 向不同流水线发射多条指令，实现**双发射甚至多发射**。例如，一个 scheduler 可以将浮点运算指令发射到 FPU，同时另一个 scheduler 将内存加载指令发射到 LD/ST 单元。

3.3 依赖追踪：从 Scoreboard 到编译器控制位

3.3.1 Scoreboard 机制

在"Two Loop" 模型中，一个 warp 可能同时有多条指令处于 in-flight 状态。为了在保证按序发射的前提下检测数据冒险（RAW, WAW），GPU 使用 **Scoreboard（记分板）**来追踪寄存器的忙闲状态。

Scoreboard 本质上是一个位图（bit-array），每一位对应一个寄存器。如果某位为 1，表示有已发射但未完成的指令将要写入该寄存器；后续依赖该寄存器的指令必须等待。由于 GPU 的寄存器规模极其庞大（64 warp × 128 寄存器 = 8192 个），如果为所有寄存器维护一个统一的大 scoreboard，并允许所有 warp 的指令每个周期都去查询，会带来噩梦般的端口开销（理论上可能需要 256 个端口）。

NVIDIA 的实际解决方案非常精巧：首先是**缩小 scoreboard 规模**，每个 warp 的 scoreboard 不再用 128 位位图追踪所有寄存器，而是只用一个很小的表（如 6 个 entry）记录当前 busy 的寄存器编号。其次是**一次性依赖检查**，在指令译码并存入 I-Buffer 的那一刻，硬件将指令的源寄存器与 scoreboard 小表进行关联查找。如果命中，就在指令的依赖位向量中设置对应位。最后是**广播唤醒机制**，当某条指令完成写回时，它会广播"某 scoreboard 槽位已释放" 的信号，I-Buffer 中所有等待该槽位的指令会自动清除自己的依赖位，从而变为就绪状态。

这种设计将原本需要持续轮询的集中式 scoreboard，转化为分布式的”入队时检查+完成时广播唤醒”机制，完美解决了大规模并行下的依赖追踪难题。

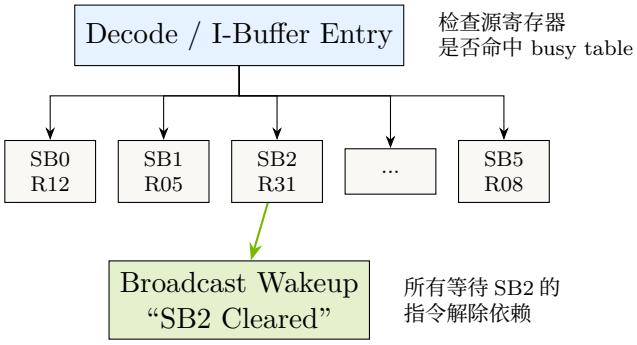

图 4: Per-warp Scoreboard 的广播唤醒机制。指令译码时与 6-entry busy table 进行关联检查，写回完成时通过广播唤醒等待该 SB entry 的指令。

3.3.2 编译器控制位（Control Bits）：NV 的软硬件协同设计

从 Kepler 架构开始，NVIDIA 进一步将部分延迟管理的复杂性从硬件迁移到了编译器。编译器在进行静态分析时，完全清楚每条指令的流水线延迟和数据依赖关系。因此，NVCC 会在生成的机器码（SASS）中插入特殊的**控制位（control bits）**，作为编译器写给硬件调度器的”小纸条”，直接指导发射调度。

这些控制位主要包括：**Stall Counter** 针对固定延迟指令（如 ALU 运算），编译器预设一个周期计数，硬件在发射该指令后会自动阻塞对应 warp 指定周期数，从而实现精确的 producer-consumer 延迟管理；**Yield 位**作为编译器引导的调度提示，当该位被设置时，warp 会主动让出发射槽位给其他 warp，以避免结构冒险和寄存器文件 bank conflict；**Dependence Counters（SB0-SB5）**针对可变延迟指令（如内存访问），编译器通过 DAG 依赖图分配六个计数器，实现对细粒度依赖链的跟踪；以及 **DEPBAR.LE 指令**提供显式的可复用等待/栅栏机制，支持部分等待和 SB 计数器的复用。

这意味着，现代 NVIDIA GPU 实际上采用了一种**硬件-编译器协同设计（hardware-compiler co-design）**的调度哲学：编译器的静态分析替代了传统硬件 scoreboard 的大量动态检查工作。相关研究表明，现代 NVIDIA GPU 的 issue scheduler 采用了 **CGGTY（Compiler Guided Greedy Then Youngest）**策略，其中编译器提供的 stall/yield/dependence 控制位是调度决策的核心依据。这一发现也解释了为什么 NVIDIA 的闭源编译器（NVCC）在性能上往往显著优于第三方编译器或手写 PTX —— 因为它掌握了与底层硬件之间的”私有协议”。

3.4 分支处理与控制流演进

3.4.1 Pre-Volta：SIMT Stack 与死锁问题

在 Volta 之前的架构中，GPU 使用 **SIMT Stack** 来处理分支发散。当 warp 遇到分支指令时，硬件会将当前执行路径（包括程序计数器 PC 和活跃线程掩码 active mask）压入一个后进先出的栈中，然后先执行一个分支路径，待该路径所有线程到达汇合点

（reconvergence point，通常是 IPDom，即 Immediate Post-Dominator）后，再从栈中弹出另一条路径继续执行。

这种机制虽然硬件实现简单，但存在一个根本性问题：**强制串行化和汇合点约束可能导致死锁**。典型场景是自旋锁（spinlock）：假设一个 warp 中部分线程尝试获取锁（进入自旋），另一部分线程已经持有锁并准备释放。由于 SIMT Stack 要求所有线程必须先完成当前路径并到达 IPDom 才能切换，而持有锁的线程在释放前被阻塞在等待自旋线程汇合的位置，自旋线程又在等待锁被释放，从而形成了死锁。

3.4.2 Post-Volta：显式同步指令与 Hanoi 微架构

从 Volta/Turing 开始，NVIDIA 彻底重构了控制流机制。新的设计不再依赖硬件隐式的 SIMT Stack，而是引入了一套**显式同步指令集**：`BSSY`（分支同步设置）、`BSYNC`（分支同步等待）、`BREAK`、`BMOV`、`YIELD`、`WARPSYNC` 等。这些指令允许编译器显式地管理线程分化与重新汇合的细节，路径可以交错执行，汇合点也可以由编译器灵活设置。

支持这些新语义的是一种被称为 **Hanoi 微架构**的低成本硬件设计。其核心组件包括 **Warp Split Stack** 记录当前活跃的分支路径片段；**Reconvergence Stack** 管理未来的汇合点；以及 **Bx 寄存器**保存分支相关的同步状态。这种架构将控制流管理的灵活性大幅提升，同时通过编译器-硬件协同避免了 SIMT Stack 的固有死锁问题。值得注意的是，AMD GPU 采用的是另一种方案：**Exec Mask**。AMD 使用一个位掩码来表示当前 warp 中哪些线程是活跃的，分支处理通过修改 exec mask 实现。这种方案硬件更简洁，但编译器负担更重，且重新汇合的灵活性不如 NVIDIA 的 Hanoi 设计。

3.5 寄存器文件设计与 Operand Collector

3.5.1 寄存器文件面临的挑战

现代 GPU 的寄存器文件（Register File, RF）是一个工程奇迹。以 Ampere/Hopper 为例，每个 SM 拥有 **256KB 的寄存器文件**。之所以需要如此巨大的 RF，是因为 GPU 要求所有常驻 warp 的上下文必须**常驻片上**，以实现零开销的 warp 切换。这与 CPU 的上下文切换机制形成鲜明对比：CPU 在切换线程时需要将寄存器状态保存到内存，再加载新状态，开销较大。

巨大的 RF 带来了两个核心挑战。首先是**面积与功耗**：256KB 的 SRAM 占据了 SM 中可观的芯片面积，并产生大量静态功耗。其次是**端口压力**：一条 FMA 指令需要同时读取 3 个源操作数并写入 1 个目标操作数。如果 SM 支持双发射，一个周期可能需要 8 个以上的读取端口。存储器的面积和端口数呈超线性增长，直接设计一个 64 端口的 256KB SRAM 几乎是不可能的。

3.5.2 Register File Banking 与 Operand Collector

解决方案是将寄存器文件**分块（banking）**。RF 被物理划分为多个独立的小 bank，每个 bank 拥有自己的读写端口。不同的 warp 或不同的操作数请求会被分散到不同的

bank，从而实现并行访问。banking 不仅是逻辑上的分区，更是**物理硬件上的独立存储块**，这显著降低了单个大存储模块的访问功耗和延迟。

但 banking 引入了新的挑战：**bank conflict**。如果两条待发射的指令在同一个周期需要访问同一个 bank，至少有一个请求必须被延迟。为了仲裁和解决 bank conflict，NVIDIA 在 Issue Scheduler 和执行单元之间引入了一个专门的 **Register Read Scheduler（也称 Operand Collector）**。该调度器负责接收来自发射调度器的多个操作数读取请求，分析每个请求需要访问的 RF bank，在每个周期内尽量将无冲突的请求分发到各个 bank，对于发生冲突的请求则进行排队和重调度。

NVIDIA 在 Fermi 架构中就采用了 operand collector 设计，通过多周期的操作数收集过程，将原本需要同时完成的多个 RF 访问分散到多个周期，从而用更少的物理端口支持更高的指令发射率。

3.5.3 寄存器层次结构与复用优化

学术界曾提出过为 GPU 设计**三级寄存器层次结构**（MRF - ORF - LRF）的方案，将短生命周期的寄存器值存储在靠近 ALU 的小型缓存中，以减少对主寄存器文件（MRF）的访问。研究表明，约 50%的 produced values 在 3 条指令内只被读取一次，非常适合放入小型近端存储。

但现代 NVIDIA GPU 并未采用这种复杂的硬件缓存方案，而是选择了另一条路：**编译器控制的复用优化**。通过寄存器重用位（reuse bits）、固定延迟设计、以及 bypass 机制，NV 优先保证行为的可预测性和静态可分析性，将复杂性再次推向了编译器。例如，编译器可以通过操作数复用分析，尽量减少对 RF 的重复访问；通过调度指令顺序，最大化利用 ALU 内部的 bypass 网络。

4 Tensor Core 架构演进与软硬件协同

4.1 从 CUDA Core 到 Tensor Core：动机与背景

GPU 最初是为图形渲染而设计的，其 CUDA Core 擅长执行标量或向量运算（如 FP32 FMA）。然而，随着深度学习时代的到来，计算模式发生了根本性转变：神经网络的训练和推理核心负载是**大规模的矩阵乘法（GEMM）**。在传统的 CUDA Core 上，一个 $D = A \times B + C$ 的矩阵运算需要分解成海量的独立 FMA 指令，每个指令从寄存器文件中读取三个操作数，经过 ALU 计算后写回结果。这种模式暴露了两个瓶颈：一是**指令开销**，每个 FMA 都需要取指、译码、发射、执行，大量的控制开销被浪费在重复做同一件事上；二是**数据搬运**，整个矩阵乘法过程中，数据吞吐量受限于 CUDA Core 的读取/写回能力，而实际计算量/数据量之比（Arithmetic Intensity）并不低。

根据 Amdahl 定律，如果程序中有 90%的时间花在矩阵乘上，那么即使把这部分加速 10 倍，整体也只能获得约 5 倍的提升。因此，在 NVIDIA V100（Volta 架构）之前，GPU 在深度学习工作负载上的效率已经遇到了明显的天花板。NVIDIA 的解决方案是在 SM 内部增加一种**领域专用加速器（DSA）**：Tensor Core。

4.2 Volta：第一代 Tensor Core（Warp-scoped MMA）

2017 年发布的 NVIDIA Volta 架构首次引入了 Tensor Core。第一代 Tensor Core 的设计 Philosophy 可以概括为：**在 warp 级别（32 线程）协同执行一次矩阵乘累加操作**。具体而言，对于 V100，一次 MMA（Matrix Multiply Accumulate）操作的尺寸为：FP16 精度下 $A(8 \times 4) \times B(4 \times 8) + C(8 \times 8) = D(8 \times 8)$，每个 warp 的 32 个线程共同参与，每个线程持有结果矩阵 D 的两个元素。

这一代 Tensor Core 有几个关键特点。第一是**粗粒度同步执行**：整个 warp 必须同步发射同一条 `HMMA` 指令。32 个线程像 SIMD 单元一样协同工作，但保留了 SIMT 的线程标识和地址计算能力。第二是 **Quad-pair 执行单元**：Tensor Core 在硬件上由多个 Quad-pair（每对包含两个 Quad，每个 Quad 含 4 个线程）构成运算阵列，数据在 Quad-pair 之间通过 crossbar 或固定连接传递，形成特定的 dot-product 运算模式。第三是**输入为 FP16，累加为 FP32**：为了兼顾速度和精度，Volta 采用混合精度设计。

从编程模型上看，Volta 的 Tensor Core 需要通过 `WMMA`（Warp-level Matrix Multiply Accumulate）API 调用。开发者需要显式地加载矩阵片段到 warp 级别的寄存器中，执行 MMA，再将结果存储回全局内存。

4.3 Turing：低精度扩展与第二代 Tensor Core

Turing 架构（2018）对 Tensor Core 的改进主要集中在**精度支持的扩展**上，以配合推理场景对速度和功耗的极致追求。Turing 在保留 FP16/FP32 混合精度的同时，增加了 INT8 精度支持（峰值算力相比 FP16 翻倍）、INT4 精度支持（峰值算力再翻倍）、以及 INT1 位运算支持（用于 DLSS 等场景）。

此外，Turing 还引入了**结构化稀疏性（Structured Sparsity）**的硬件基础，尽管在消费级产品上稀疏性的大规模应用是在 Ampere 上才真正落地。这一代 Tensor Core 的主要价值在于为**低精度推理**打开了大门。这也证明了一个重要的演进方向：不断降低运算精度，以指数级提升峰值算力。

4.4 Ampere：异步拷贝、稀疏性与第三代 Tensor Core

2020 年发布的 Ampere 架构（A100）是 Tensor Core 发展史上的一个重要里程碑。它不仅带来了第三代 Tensor Core，还首次引入了对整个 AI 训练/推理流程至关重要的新特性。

4.4.1 第三代 Tensor Core 核心能力

BF16/TF32 支持：为了满足训练对数值稳定性的需求，Ampere 增加了 Brain Floating Point 16（BF16）和 TensorFloat-32（TF32）格式。TF32 是一种 19-bit 格式（1 符号位+8 指数位+10 尾数位），在不改变数据布局的情况下，通过硬件截断实现接近 FP32 的精度和接近 FP16 的速度，是 AI 训练的理想选择。

结构化 2:4 稀疏性（Structured Sparsity）：Ampere 首次在硬件上完整支持 2:4 结构稀疏矩阵乘法。在这种模式下，矩阵中每 4 个元素中有 2 个被置零，硬件在执行 MMA 时跳过这些零元素，将有效算力提升一倍。NVIDIA 提供了相应的稀疏矩阵压缩算法和 cuSPARSELt 库支持。

更大尺寸的 MMA：warp-level MMA 的矩阵尺寸进一步扩大，使得每次指令能够处理更多的数据，提高了指令效率。

4.4.2 异步数据拷贝（cp.async）

Ampere 引入的 `cp.async`（copy asynchronous）指令，允许 GPU 在不占用 CUDA Core 的情况下，将数据从全局内存直接拷贝到共享内存。这意味着加载数据与 Tensor Core 计算可以**完全重叠**；开发者可以通过**双缓冲（double buffering）**甚至**多缓冲（multi-stage pipeline）**技术，将内存延迟隐藏在计算之中。

`cp.async` 本质上是对内存子系统纹理加载流水线（Texture Load Pipeline）的通用化，使得全局内存到共享内存的通路也可以被显式地流水线化调度。这是 Ampere 之后 NVIDIA 编程模型向**更强的异步性**演进的开端。

4.5 Hopper：WGMMA、TMA 与第四代 Tensor Core

2022 年发布的 Hopper 架构（H100）将 Tensor Core 推向了第四个世代。这一代的设计关键词是：**扩大协作粒度、增强异步能力、进一步降低精度**。

4.5.1 Warpgroup-level MMA（WGMMA）

与前三代不同，Hopper 引入了 **WGMMA（Warpgroup-level Matrix Multiply Accumulate）**。一个 warpgroup 包含 4 个 warp（共 128 线程），它们协同执行一次 MMA 指令。通过扩大矩阵运算的协作粒度，Hopper 的 Tensor Core 能够处理更大尺寸的矩阵块，减少了将大矩阵分块后迭代执行的次数，从而降低了指令发射和控制开销。

WGMMA 指令（在 PTX 中形如 `wgmma.mma_async`）的语义更加异步：Tensor Core 在后台执行矩阵乘法，而 warpgroup 中的 warp 可以在不同程度上继续执行其他指令（例如地址计算、barrier 操作等）。这种异步语义是后续 Blackwell 进一步解放编程模型的关键铺垫。

4.5.2 Tensor Memory Accelerator（TMA）

TMA 是 Hopper 架构中一个非常重要但常被低估的硬件单元。它是一个**独立的硬件引擎**，专门负责在全局内存、共享内存和 Tensor Core 的寄存器之间进行多维张量的数据搬运。TMA 的核心能力包括：**多维下标计算**，开发者只需描述张量的形状、切片大小和步长，TMA 硬件自动完成复杂的多维地址计算；**异步执行**，TMA 操作与 CUDA Core/Tensor Core 计算完全异步，可以独立地在后台运行；以及**支持集群间通信**，Hopper

引入了 Thread Block Cluster（线程块簇）编程模型，一个 cluster 内的多个 thread block 可以通过 TMA 访问彼此的共享内存，极大地简化了多 SM 协作 GEMM 的编程复杂度。

TMA 不仅减轻了线程在地址计算上的负担，还使得 GEMM 核心的**计算-加载比例**得以最大化。

4.5.3 FP8 与 Transformer Engine

Hopper 增加了对 FP8（E4M3 和 E5M2 两种格式）的硬件支持。FP8 相比 FP16，在数据位宽上又减少了一半，意味着在相同的 HBM 带宽下可以传输两倍的数据量，Tensor Core 的峰值算力也相应翻倍。H100 的稀疏 FP8 峰值算力可达近 4000 TFLOPS。

配合 FP8，NVIDIA 还推出了 **Transformer Engine**。这是一个软硬件协同的解决方案：在前向/反向传播过程中，底层框架（如 cuDNN、cuBLAS）自动地对每一层的输入/激活值进行动态缩放（dynamic scaling），找到合适的 FP8 表示范围，既充分利用 FP8 的速度优势，又避免数值溢出和下溢。

4.6 Blackwell：第五代 Tensor Core 与 TMEM 架构分析

2024 年发布的 Blackwell 架构（B100/B200）代表了 NVIDIA Tensor Core 设计的最新形态。虽然具体的微架构细节尚未完全公开，但基于现有技术文档、专利分析和 PTX 指令集的演进，可以对其核心创新进行合理推断。

4.6.1 Tensor Memory（TMEM）

Blackwell 最大的架构级变化是引入了 **Tensor Memory（TMEM）**，并将部分 MMA 操作数的存储从传统的寄存器文件（RF）转移到了 TMEM 中。TMEM 在物理上是一块**独立的、高带宽的片上存储**，专门为 Tensor Core 的数据保留。它的优势在于：专用端口提供了更高的带宽，TMEM 不需要与普通的标量/向量运算共享 RF 端口，减少了寄存器文件 banking 的压力；更长的向量生命期，对于大尺寸的 GEMM kernel，累加矩阵（accumulator）往往在多个 MMA 指令中复用，存储在 TMEM 中可以减少向 RF 的来回搬运；以及**单线程 MMA 语义**，PTX 中出现了 `tcgen05.mma` 这样的指令，语义上看起来似乎是 Thread 级别的（而非 warp-level）Tensor Core 操作。这意味着 Thread 作为独立的实体发起 Tensor Core 请求，硬件在底层重新组合成脉动阵列（systolic array）的调度模式，编程模型进一步解耦。

4.6.2 CTA Pair 与双 SM 协作

Blackwell 支持 **CTA Pair（Cooperative Thread Array Pair）**，即两个相邻的 SM 可以作为一个更大的逻辑单元协作执行。两个 SM 的 Tensor Core 资源可以汇总，处理更大规模的矩阵块。这相当于在 SM 之上增加了一个新的层级：**Pair of SMs**，为未来的超大模型推理和训练提供了更强的单卡扩展性。

4.6.3 精度进一步降低：MXFP 与 NVFP4

Blackwell 支持 **MXFP6/MXFP4**（Microscaling Floating Point）以及 **NVFP4**。这些格式通过共享一个块的 exponent scale factor，进一步压缩了数据表示的位宽，同时尽量保持模型的精度。这标志着 NVIDIA 在" 降低精度换取算力" 这条路上已经走到了极致边缘。

4.6.4 对 Blackwell 架构的批判性分析

尽管 Blackwell 在峰值算力和异步化程度上取得了显著进步，但基于业内资深架构师的分析，这一代产品也存在不容忽视的设计瑕疵。最大的争议来自 **B200 上 SFU（Special Function Unit）与 Tensor Core 算力不匹配**的问题。为了在单个 Die 上容纳大量的 TMEM、GPC 互联网络（DSMEM）以及取消分区的 L2 缓存，B200 单个 Die 上的 SM 数量被压缩到了约 80 个。然而，CUDA Core 配套的 SFU 性能并没有同步增强，而 Transformer Attention 中的 Softmax 运算恰恰重度依赖 SFU（指数、倒数等超越函数）。这导致在实际 LLM 推理中，当 GEMM 被 Tensor Core 大幅加速后，Softmax 反而成为了新的瓶颈。NVIDIA 在 **B300** 上通过**削减 FP64 和 INT8 算力**的方式腾出了芯片面积来增强 SFU，某种程度上印证了 B200 在这方面的不足。

其次，Blackwell 的**指令结构显著复杂化**。`tcgen05` 系列指令既包含同步操作（`alloc/dealloc/fenc`
又包含异步操作（`mma/cp/shift/ld/st`），而且这些指令的协作粒度参差不齐：有的是线程级别，有的是 warp 级别，还需要考虑 2-CTA Pair 的情况。虽然 NVIDIA 通过 Pipeline 抽象和 TMEM 的基于列（column-based）的分配/释放机制缓解了部分同步编程的困难，但这无疑抬高了手写高性能 kernel 的门槛。正如批评者指出的，从 Volta 到 Blackwell 的十年演进中，NVIDIA 真正的壁垒恰恰在于把这些"dirty work" 处理得足够干净，在易用性与性能之间取得了微妙的平衡，但 Blackwell 的复杂度已经逼近了这个平衡的临界点。

最后，Blackwell 采用的 **Dual-Die 架构**也带来了跨 Die 内存访问延迟的问题。当 SM 需要访问位于另一 Die 上的 HBM 时，延迟明显高于本 Die 访问，而当前 CUDA 版本尚未提供 CTA 级别的内存亲和性调度 API，这可能影响部分 GMEM 密集型 kernel 的实际效率。

4.7 Tensor Core 演进趋势总结

表 1 总结了 Tensor Core 从 Volta 到 Blackwell 的代际演进。

表 1: Tensor Core 代际演进对比

特性	Volta	Turing	Ampere	Hopper	Blackwell
代数	1st	2nd	3rd	4th	5th
协作粒度	Warp (32)	Warp (32)	Warp (32)	Warpgroup (128)	CTA Pair / Thr
核心指令	`wmma`	`wmma`	`wgmma` (legacy)	`wgmma_async`	`tcgen05.mma`
异步加载	无	无	`cp.async`	TMA + Async	TMA + TME
精度支持	FP16+FP32	FP16/INT8/INT4	+BF16/TF32/稀疏	+FP8	+MXFP/NVF

从上表可以看出，Tensor Core 的演进呈现出几个明确的趋势。第一是**协作粒度不断扩大**：从 warp 到 warpgroup，再到 CTA pair，NVIDIA 试图在单个指令中塞入更多的运算，以减少指令发射和控制开销。第二是**异步化持续提升**：数据搬运（cp.async 到 TMA）和计算（sync MMA 到 async MMA）的异步程度越来越高，CUDA 编程模型从"同步 SIMT"向"异步数据流"演进。第三是**精度持续降低**：从 FP16 到 INT8/INT4 到 BF16/TF32 到 FP8 到 MXFP4，精度每降低一档，峰值算力就大致翻倍，硬件设计也越来越依赖软件层的动态缩放和量化补偿。第四是**存储层次复杂化**：操作数存储从寄存器文件（RF），到共享内存，再到 TMEM，显示了 NVIDIA 对数据局部性和 RF 端口瓶颈的回应。

5 CuTe Tile 编程与 Layout 代数

5.1 CuTe 的架构定位与编程动机

前文对 Tensor Core 与异步数据搬运硬件机制的论述，揭示了现代 NVIDIA GPU 在微架构层面的高度复杂性。然而，硬件能力的释放最终依赖于 software stack 能否以高效且可维护的方式将这些能力暴露给程序员。在 CUTLASS 2.x 时代，开发者需要手写大量基于模板元编程（template metaprogramming）的索引计算和 warp-level 划分逻辑，代码冗长且难以跨架构复用。为了解决这一问题，NVIDIA 在 CUTLASS 3.x 中引入了 **CuTe（CUDA Tensor）**——一套基于 **Layout 代数（Layout Algebra）**的 C++ DSL，旨在将 Tile 编程中的索引计算、线程-数据映射、以及 MMA 指令调度转化为可组合、类型安全、且与底层 PTX 解耦的泛型操作。

CuTe 的核心设计哲学可以概括为：**将易错的索引算术替换为 Layout 的组合运算**。在传统的手写 CUDA kernel 中，开发者需要手动计算每个线程应该读取的全局内存偏移、共享内存中的 bank 对齐地址、以及 Tensor Core 寄存器分片的索引。这些计算往往涉及大量的乘加、取模和条件分支，稍不注意就会引入 coalescing 失败、bank conflict 或 MMA 数据错位。CuTe 通过引入统一的 `Shape`、`Stride`、`Layout` 和 `Tensor` 抽象，把这些问题转化为高阶的代数运算（如 composition、division、zipping），从而在保证零开销抽象的同时，显著提升 kernel 的可读性和可移植性。

5.2 Layout 代数基础：Shape、Stride 与 Tensor

在 CuTe 中，一切空间映射都可以通过 **Layout = Shape : Stride** 来表达。一个 Layout 描述的是从多维逻辑坐标到一维线性内存索引的映射关系。以典型的行主序矩阵为例，其 Layout 可写作 $(N, M) : (M, 1)$，表示坐标 (n, m) 对应的线性偏移为 $n \times M + m \times 1$。

CuTe 的 `Shape` 和 `Stride` 支持任意层次的嵌套，这使得复杂的内存布局（如 bank-conflict-free 的 swizzle layout）能够被紧凑地描述。例如，一个 2D swizzle 布局可以表示为 `((2, 2), (2, 2)) : ((1, 4), (2, 8))`，其中内层的 $(2, 2)$ 描述了 tile 内部的子

块排列，外层的 $(2, 2)$ 描述了 tile 之间的宏观排列。相比手写位运算实现 swizzle，嵌套 Layout 的组合不仅更易验证，还能在编译期完成类型推导和优化。

Tensor 是 Layout 代数在 GPU 上的直接载体。一个 `cute::Tensor` 本质上是指针（或引用）加上一个 Layout。它不拥有内存，但提供了基于 layout 的切片、重排、分区等操作。关键运算包括：Composition ($B \circ A$) 将两个 layout 链式组合，是 tiling 操作的基础；Complement / Division ($B \oslash A$) 将一个 layout 分解为 intra-tile（块内）和 inter-tile（块间）两部分；`zipped_divide` 将逻辑分块重新分组为 $(\text{tile_shape}, \text{rest})$ 的二元结构；以及 `local_tile`，基于 `zipped_divide` 的封装，用于从全局 tensor 中提取当前 CTA 负责的数据块。

这些运算使得 GEMM 中的核心操作 —— 将全局矩阵切分为 CTA tile、再切分为 warp tile、再切分为 per-thread register fragment —— 可以逐层用 layout 的组合来表达，而无需显式计算偏移地址。

5.3 Tile 编程模型：Global → Shared → Register

CuTe 的 tile 编程模型遵循一条清晰的内存层级路径。一个典型的 GEMM kernel 首先定义 CTA 级别的 tile 大小：

```
auto cta_tiler = make_shape(Int<128>{}, Int<128>{}, Int<32>{}); // (
    BLK_M, BLK_N, BLK_K)
```

随后，通过 `local_tile` 从全局内存张量中提取当前 CTA 对应的数据块：

```
Tensor gA = local_tile(mA, cta_tiler, cta_coord, Step<_1, X, _1>{});
    // (BLK_M, BLK_K, k)
```

这里的 `Step<_1, X, _1>` 表示在 M 和 K 维度上按 tile 步进，而在 N 维度上忽略（因为 A 矩阵只参与 M-K 乘积）。`k` 模式索引了需要在主循环中逐个累加的 K tile。

在将数据从全局内存搬运到共享内存时，CuTe 使用 `local_partition` 定义线程布局（thread layout），并将 tile 划分为 per-thread 的子块：

```
auto tA = make_layout(make_shape(Int<32>{}, Int<8>{})); // (m, k) ->
    thr_idx
Tensor tAgA = local_partition(gA, tA, threadIdx.x); // (THR_M, THR_K,
     k)
Tensor tAsA = local_partition(sA, tA, threadIdx.x); // (THR_M, THR_K)
```

每个线程获得的子块大小由 thread layout 决定，并通过重复覆盖整个 CTA tile。当执行 `copy(tAgA(_, _, k_tile), tAsA)` 时，CuTe 会自动保证线程间的内存访问是大致 coalesced 的，并生成对应于异步拷贝（如 `cp.async`）的高效 PTX。

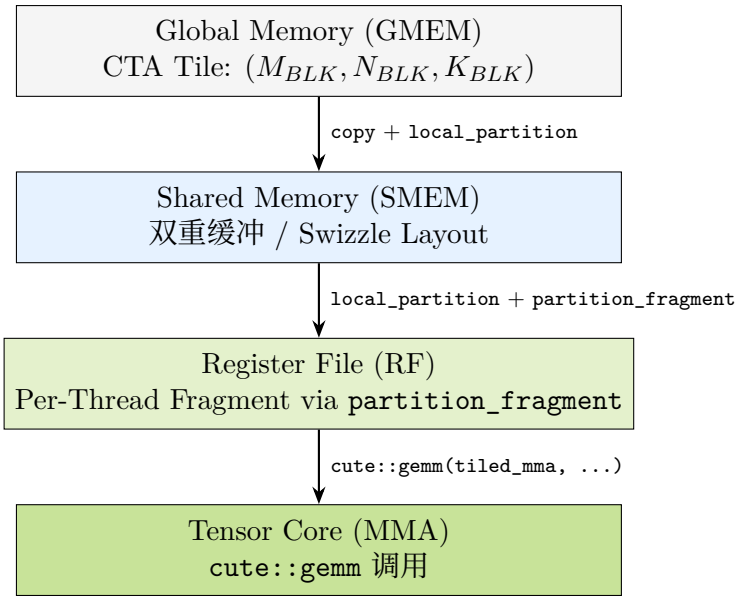

图 5: CuTe Tile 编程的数据流示意图。数据从全局内存经 `local_tile` 和 `local_partition` 逐层下放到共享内存、寄存器，最终由 `cute::gemm` 驱动 Tensor Core 计算。

计算阶段需要另一套线程布局 `tC` 来 partitioning accumulator。开发者分配与 partitioned output 形状匹配的寄存器张量作为累加器：

```
auto tC = make_layout(make_shape(Int<16>{}, Int<16>{}));
Tensor tCrC = make_tensor_like(local_partition(gC, tC, threadIdx.x));
clear(tCrC);
```

主循环中，线程首先将当前 K tile 的数据从 global 拷贝到 shared memory，同步后执行 `gemm(tiled_mma, tCrC, tArA, tBrB, tCrC)`，将累加结果保留在寄存器文件中。

5.4 MMA_Atom、TiledMMA 与 ThrMMA 的抽象层次

CuTe 对 Tensor Core 指令的封装分为三个递进的抽象层级：**MMA_Atom**、**TiledMMA** 和 **ThrMMA**。

MMA_Atom 是最底层的硬件指令抽象。它把一条具体的 PTX MMA 指令（如 `SM80_16x8x16_F16F16F16F16_TN`）包装为两部分：一是暴露原始 PTX 接口的 `Operation`，二是包含元信息的 `MMA_Traits`。Traits 中定义了该指令的逻辑数据类型 `ValTypeD/A/B/C`、逻辑形状 `Shape_MNK`，以及最关键的线程-值映射 `ALayout`、`BLayout` 和 `CLayout`。这些 Layout 直接编码了 $(\text{thread}, \text{value}) \to (m, k)$ 或 (m, n) 的映射关系。例如，对于 Volta 的 `SM70_8x8x4_F32F16F16F32_NT`，其 `CLayout` 可以表示为嵌套的 `Layout<Shape, Stride>`，把 8 个线程和 8 个值映射到 8×8 的结果矩阵坐标上。

TiledMMA 则是在 `MMA_Atom` 之上的组合器。通过 `make_tiled_mma`，开发者能够将单个 Atom 在 M、N、K 维度上进行复制和排列，以适应更大的 tile 尺寸。`TiledMMA` 的模板参数包括 Atom layout（描述 Atom 在线程间的排列，如 `Layout<Shape<_2, _2>, Stride<_2, _1>>`）和可选的 permutation tiler。例如，将 4 个 Atom 拼成一个覆盖 16×16

矩阵的 warpgroup-level MMA，只需在外部叠加一个 2×2 的 Atom layout，而无需修改底层 PTX 调用。

```
using MMA_Atom_Arch = MMA_Atom<MMA_Traits<
   SM80_16x8x16_F16F16F16F16_TN>>;
auto tiled_mma = make_tiled_mma(MMA_Atom_Arch{},
                                make_layout(Shape<_2, _2, _1>{}),
                                make_tile(_32, _32, _16));
```

ThrMMA 是 `tiled_mma.get_thread_slice(threadIdx.x)` 返回的 per-thread 视图。它利用 TiledMMA 的组合 layout，自动推导出当前线程在全局/共享张量中应该读取的寄存器 fragment 的位置和形状。通过 `partition_A`、`partition_B`、`partition_fragment_A` 等接口，开发者可以直接获得当前线程需要参与 MMA 的所有源操作数和累加器的 `Tensor` 视图，而不必手动计算寄存器分配表。

5.5 CuTe 的架构意义

CuTe 的出现标志着 NVIDIA 在高性能 GPU kernel 开发范式上的一次重要转向：从手写索引算术和模板元编程，转向基于 **Layout 代数的声明式编程**。这一转变具有深远的软硬件协同意义。

首先，CuTe 将 Tensor Core 的线程-数据映射完全参数化。不同架构（Volta、Ampere、Hopper、Blackwell）的 MMA 指令具有迥异的数据排布和协作粒度，但在 CuTe 中，它们都统一表现为具有不同 `MMA_Traits` 的 `MMA_Atom`。这意味着同一个基于 CuTe 编写的 GEMM kernel 可以在不同 GPU 代际之间实现更高程度的代码复用，开发者只需调整 `TiledMMA` 的参数和 tile 大小，而无需重写核心的索引计算逻辑。

其次，CuTe 把异步编程模型（如 TMA、`cp.async`、WGMMA）自然地融入了 Layout 代数框架。异步拷贝的线程划分、TMA 的 cluster-level 张量切片、以及 WGMMA 的 warpgroup 级累加器分布，本质上都是不同层级上 layout 的分区与组合问题。CuTe 为这些操作提供了统一的抽象接口，使得 Hopper/Blackwell 上复杂的异步流水线 kernel 可以被更简洁、更可靠地表达。

最后，CuTe 代表了 GPU 编译器-硬件协同设计的又一深化。正如第 3 章所述，NVIDIA 将 SASS 层面的 control bits 作为编译器与硬件之间的”私有协议”。CuTe 则在更高的中间层（C++ DSL / PTXAS 层面）上建立起另一层协议：它既向程序员暴露了足够的高阶抽象以简化开发，又向下翻译成了足够底层的、可高度优化的 PTX/CUDA C++ 代码。随着 AI 模型对自定义 kernel（如各种变种的 FlashAttention、MoE routing kernel）需求的激增，CuTe 这样的抽象层将成为连接算法创新与硬件性能的关键桥梁。

6 内存层次与数据移动优化

6.1 GPU 内存层次结构全景

现代 GPU 拥有一套精心设计的多级内存层次，旨在在不同的容量、带宽和延迟之间取得平衡。从高到低（从程序员可见到微架构底层），可以概括为：寄存器文件（Register File, RF）每个线程独占，延迟极低（∼1 周期），带宽极高；共享内存/L1 缓存（Shared Memory / L1D Cache）同 block 内线程共享，延迟约 10-30 周期；L2 缓存（Unified L2 Cache）所有 SM 共享，延迟约 100-200 周期；全局内存/HBM（High Bandwidth Memory）最大容量（数十 GB），延迟可达数百甚至上千周期，但带宽极高（H100 达 3-4TB/s）。

这种层次结构体现了一个核心设计哲学：**把编程控制权和硬件优化机会留给离计算单元最近的地方**。程序员可以通过`__shared__`关键字精确控制数据在共享内存中的布局；编译器负责寄存器分配；而 L2 缓存的替换策略和预取行为则由硬件自动管理。

6.2 寄存器文件设计：不仅仅是 SRAM

寄存器文件在 GPU 中的角色远比 CPU 中复杂。前文在 SM 微架构部分已经提到，RF 采用了 banked 设计和 operand collector 机制来缓解端口压力。但除此之外，RF 的设计还涉及以下关键技术。

现代 GPU 的 RF 通常支持**超宽访问**。例如，一次读取可以返回一个 128-bit 或 256-bit 的向量寄存器值。这允许在 FMA 指令中一次性读足 3 个操作数，而不需要拆分成多次标量读取。向量读写端口的共享，是 operand collector 调度策略中的一个重要优化维度。

为了减少刚计算出的结果被写回 RF 后又立即读出的开销，执行单元之间通常设置了 **bypass 网络**。如果某条指令的操作数恰好是前一条指令刚产生的结果，operand collector 可以跳过 RF 读取，直接从 ALU 的输出端获取数据。这既是编译器调度优化的目标之一，也是硬件设计的固有特性。

6.3 共享内存与 Bank Conflict

共享内存（Shared Memory）是 GPU 编程中进行高性能数据复用的关键资源。它之所以被称为”可编程缓存”，是因为程序员必须显式将全局内存的数据加载到共享内存，并且需要设计数据的存储方式，以最大化并行访问效率。

共享内存在物理上被划分为多个 bank（通常是 32 个 bank）。理想情况下，warp 中的 32 个线程在同一个周期分别访问不同的 bank，硬件可以在一个周期内并行完成所有访问。但如果多个线程访问了同一个 bank（且地址不同），就会发生 **bank conflict**，访问会被串行化，性能大打折扣。

为了规避 bank conflict，高性能 GEMM kernel（如 CUTLASS 中的实现）通常会精心设计矩阵在共享内存中的 **swizzle layout**，通过地址计算将原本冲突的访问分散到不同的 bank。

6.4 L1/L2 缓存与 Cache Line 管理

NVIDIA GPU 的 L1 Data Cache 和 Shared Memory 是**统一存储**(Unified L1D/Shared Memory)。同一块片上 SRAM 可以根据配置划分为不同比例的 L1 cache 和 Shared Memory。例如，在 Ampere 上，每个 SM 有 128KB 的统一存储，可以配置为 112KB Shared + 16KB L1，或者 48KB Shared + 80KB L1 等。

L2 cache 作为全局统一的缓存层，采用了**非阻塞（non-blocking）**设计。一个 warp 的全局内存访问如果在 L2 命中，可以直接返回；如果不命中，访问请求会被发送到 HBM，同时 warp 被切换出去执行其他指令。L2 的缺失不会阻塞整个 SM，这是 GPU 能够容忍高内存延迟的关键。

6.5 异步数据搬运：cp.async 与 TMA

在现代 NVIDIA GPU 上，数据搬运与计算的重叠优化是榨取峰值性能的核心技术。Hopper 和 Ampere 提供了两套异步数据搬运机制。

`cp.async` 指令允许从全局内存异步拷贝数据到共享内存，并通过 `cp.async.wait_group` 和 `cp.async.commit_group` 进行流水线化管理。典型的软件流水线 pattern 如下：Prologue 阶段预加载第 0 个阶段的 A/B 矩阵块到共享内存；Mainloop 阶段对第 k 阶段，用 Tensor Core 计算，同时启动第 $k+N$ 阶段的异步加载；Epilogue 阶段清空最后的异步加载，写回结果。

TMA 将异步数据搬运提升到了硬件引擎级别。相比 `cp.async` 只是发起一次普通的全局内存加载，TMA 能够自动处理多维张量的切片和边界条件，支持 cluster 级分布式共享内存访问，并与 `mbarrier`（memory barrier）机制配合，实现基于到达计数（arrival count）的精确同步。TMA 的引入，使得 CUDA kernel 在写 GEMM 时可以更加接近”纯计算声明式”编程，而不必手动处理繁琐的地址计算和 dma 描述符配置。

6.6 多 GPU 互连与内存一致性

当单卡的算力和内存容量不足以满足大模型训练需求时，需要在多 GPU 之间进行高速互连。现代多 GPU 系统的互连技术经历了从 PCIe 到 NVLink 再到 NVSwitch 的演进。

PCIe 是最基础的 GPU 互联方式，带宽通常在 32-128GB/s 量级。为了绕过 CPU 内存的额外拷贝开销，NVIDIA 推出了 **GPUDirect** 技术族，包括：GPUDirect RDMA 允许网卡或存储设备直接将数据写入 GPU 显存；GPUDirect P2P 支持两块 GPU 直接通过 PCIe 交换数据；GPUDirect Storage 允许 GPU 直接从 NVMe 存储读取数据。

NVLink 是 NVIDIA 的专用高速互连，带宽远超 PCIe。Hopper 上的第四代 NVLink 单链路可达 900GB/s（双向）。通过 **NVSwitch** 芯片，可以在 8-GPU、16-GPU 甚至 256-GPU（如 DGX H100 SuperPOD）的拓扑中实现全对全（all-to-all）的高带宽通信。NVSwitch 的引入，本质上是把 GPU 网络从基于 PCIe 的层次化树形拓扑，扩展成了更扁平的、类胖树的交换网络。

在 NVIDIA 的 **Grace-Blackwell（GB200）**超级芯片方案中，CPU 与 GPU 通过 NVLink C2C 高速互联，旨在降低数据搬运开销。然而，这一设计也受到了业内的批评。首先是**"Killer Microsecond" 问题**：随着 Tensor Core 算力越来越强，许多 kernel 的运行时间已经下降到了微秒级别，而 Grace CPU 的 kernel launch 开销相对较高，导致 CPU 成为了延迟瓶颈。其次是 **Grace 微架构本身的裁剪**：虽然 Grace 采用了 ARM Neoverse V2 核心，但其 L2 缓存被 NVIDIA 削减到了 1MB（标准 V2 为 2MB），加上片内 Mesh NOC 的多跳延迟和" 嘈杂邻居"（noisy neighbor）效应，使得 Grace 的延迟表现不如 AWS Graviton 4 等同类 ARM 服务器 CPU。此外，GB200 搭配的 CX7 网卡没有内置 PCIe Switch，导致 Scale-Out 的 RDMA 流量必须穿越 Grace 的 NOC 才能到达 Blackwell，进一步加剧了延迟和缓存 miss 的惩罚。这些问题在后续的 GB300 平台中得到了部分缓解。

在多 GPU 系统中，GPU 的内存模型是**物理上分离、逻辑上通过统一虚拟地址（UVA）和 NCCL 库抽象**的。NVIDIA 的统一内存（Unified Memory）允许 CPU 和多个 GPU 共享同一个虚拟地址空间，由硬件自动进行页面迁移（page migration）。然而，对于追求极致性能的大型分布式训练，开发者通常仍然选择显式管理数据分布（如 Megatron-LM 和 DeepSpeed 中的张量并行、流水线并行和 ZeRO 优化），以避免统一内存自动迁移带来的不可预测开销。

7 NVIDIA GPU 架构代际对比

7.1 从 Pascal 到 Blackwell：计算能力演进

自 2016 年以来，NVIDIA 的数据中心 GPU 架构经历了 Pascal 到 Volta 到 Ampere 到 Hopper 再到 Blackwell 的五代更迭。每一代都在 SM 数量、Tensor Core 能力、内存带宽和精度支持上取得了显著进步。表 2 给出了各代数据中心旗舰 GPU 的关键参数对比。

表 2: NVIDIA 数据中心 GPU 架构代际对比

架构	产品	SM 数	TC 代数	FP16 峰值	HBM 带宽	L2 缓存
Pascal	P100	56	无	∼21 T	732 GB/s	4 MB
Volta	V100	80	1st	∼125 T	900 GB/s	6 MB
Ampere	A100	108	3rd	∼312 T	2 TB/s	40 MB
Hopper	H100	132	4th	∼1000 T	3.35 TB/s	50 MB
Blackwell	B100	∼144	5th	∼3500 T	∼8 TB/s	显著增大

注：上表中的峰值算力以 *Tensor Core FP16/BF16 Dense* 计算为准，不同来源可能有差异。

7.2 SM 微架构的代际差异

从 SM 结构的角度看，每一代的变化也反映了 NVIDIA 设计哲学的微调。

Pascal 每个 SM 被划分为两个 Processing Block, 每个 block 有独立的 warp scheduler 和 register file 分区。这是 NVIDIA 在 SM 内部分块设计思维的开端。**Volta**SM 内的 Tensor Core 首次出现。Volta 将 SM 划分为 4 个 Processing Block，进一步细化了调度粒度。同时，每个 SM 增加了独立的 INT32 单元，允许 FP32 和 INT32 运算并行进行。**Ampere**SM 内的 Tensor Core 数量增加，且数据处理单元（ALU）和加载/存储单元更加充裕。每个 SM 包含 4 个 warp scheduler，每个 scheduler 配有一个 dispatch 单元。Shared/L1 的统一性和可配置性也更加灵活。**Hopper**SM 数量激增至 132 个。Tensor Core 的吞吐提升了一倍，且支持 WGMMA 这种 coarse-grained 操作。Hopper 还引入了分布式共享内存（Distributed Shared Memory），允许一个 Thread Block Cluster（最多 8 个 block）之间进行快速的 SM-to-SM 通信。**Blackwell** 基于现有信息，Blackwell 的 SM 进一步增强了 Tensor Core 的比例，CUDA Core 相对减少（因为 AI 负载对 Tensor Core 的利用率远高于 CUDA Core）。然而，由于 TMEM、DSMEM 互联网络以及统一 L2 缓存占用了大量 Die 面积，单个 Die 上的 SM 数量反而下降到了约 80 个（B200）。TMEM 的引入也意味着 SM 内部的存储层次发生了根本性变化。

7.3 精度演进与 AI 训练的匹配

Tensor Core 的精度演进与深度学习模型的发展趋势息息相关。FP16（Volta/Turing）适合早期的 CNN、RNN 训练。BF16/TF32（Ampere）是 BERT/GPT 等大模型训练的主流精度，兼顾数值稳定性和速度。FP8（Hopper）使得 Transformer 类的 LLM 训练开始大规模使用，通过 per-tensor scaling 实现稳定收敛。MXFP4/NVFP4（Blackwell）面向超大模型的推理部署，极致压缩模型权重和 KV Cache。

需要强调的是，虽然峰值算力随着精度降低而指数增长，但**有效利用率（Effective TFLOPS utilization）**并不一定同比例提升。FP8 和 FP4 需要更复杂的量化策略、动态缩放补偿，以及混合精度回退（fallback）机制。在学术和工程实践中，评估 Tensor Core 性能时不应只看峰值算力，而应关注在真实 GEMM fused kernel 中的利用率（如 FlashAttention 等典型负载）。

8 替代加速器架构对比

尽管 NVIDIA GPU 在 AI 领域占据了统治地位，但学术界和产业界从未停止对替代架构的探索。下面介绍近年来备受关注的两种非 GPU 加速器：Groq LPU 和 SambaNova SN40L RDU，并与 NVIDIA GPU 进行对比。

8.1 Groq LPU：确定性执行与张量流式处理器

Groq 推出的 Language Processing Unit（LPU）是一种极具特色的 AI 加速器。与 GPU 的" 通用并行计算+批量数据复用" 思路不同，Groq 采用了**确定性数据流架构**。

其核心设计特点包括：**SRAM 为中心**，Groq LPU 几乎没有片外 DRAM（或仅有极小容量）。整个模型被静态地划分到多个 LPU 芯片的 SRAM 中。推理时，token 在各

个芯片的 SRAM 之间按固定路径流动，避免了访问 DRAM 带来的高延迟。**时序完全确定**，Groq 架构在编译时就已经确定了每个操作在每个时钟周期发生的精确位置。运行时不需要动态调度、没有 cache miss、没有 bank conflict，一切行为都是确定的。这种**编译时调度（compile-time scheduling）**使得延迟极低且高度可预测。**张量流式处理（Tensor Streaming Processor, TSP）**，Groq 的底层执行单元采用 TSP 设计，数据在一个二维的 PE（Processing Element）阵列中以固定节律流动，类似于 systolic array。计算和通信被完美地流水线化。

表 3: NVIDIA GPU 与 Groq LPU 的对比

维度	**NVIDIA GPU**	**Groq LPU**
存储层次	HBM + L2 + Shared + RF	以 SRAM 为主，几乎无 DRAM
编程模型	CUDA（显式并行）	编译器静态映射（使用时自动编译）
延迟特征	低但可变（受调度、cache、通信影响）	极低且完全确定
吞吐特征	高 batch size 下极佳	低 batch size（甚至单 batch）下极佳
适用场景	训练 + 高并发推理	低延迟在线推理（LLM Chatbot）

Groq 的优势在于**单用户的低延迟交互式推理**（如 LLM Chat）。其公布的 Llama-2 70B 推理延迟数据在当时引发了广泛关注。然而，Groq 的挑战也十分明显：一是**模型容量受限于 SRAM**，大模型需要大量芯片扩展，硬件成本高昂；二是**不支持训练**，当前 LPU 仅支持推理，无法处理复杂的反向传播和权重更新；三是**通用性差**，架构高度特化于 Transformer 类 LLM，对其他类型 AI 任务（如 CNN、图网络）的适配性存疑。

8.2 SambaNova SN40L RDU：可重构数据流单元

SambaNova 的 DataScale 系统基于 **Reconfigurable Dataflow Unit（RDU）**构建。RDU 的核心思想是：**通过可重构的硬件数据流，将计算和存储紧密耦合，最大化数据复用**。

SN40L 采用了创新的**三级存储架构**：片上 SRAM（Tensor Memory）供 RDU tile 直接访问；片外近存计算存储（DataScale Memory）是一种高容量高带宽的存储扩展，容量可达数 TB；可选的 DDR/HBM 用于存储更大的激活或权重。RDU 的硬件由大量**可重构的并行处理单元**和**可编程互连网络**组成。编译器在编译时将神经网络的计算图直接映射到 RDU 的物理数据流路径上，权重和激活值被分配到最优的存储层级。

与 GPU 相比，**数据流的静态重构 vs SIMT 的动态调度**：GPU 的 warp scheduler 在运行时动态调度线程和数据（SIMT），而 RDU 在编译时就确定了数据从哪个存储单元流向哪个计算单元。这使得 RDU 在某些数据复用极高的工作负载上（如大型 Embedding lookup、稀疏 LLM 推理）可以达到更高的效率。**内存容量优势**：相比 GPU 受限于 HBM 容量（目前最高约 192GB），SambaNova 的 DataScale Memory 可以提供数 TB 的有效容量，对于超大模型的推理具有独特吸引力。

SambaNova 的主要挑战在于生态和软件栈成熟度。CUDA 经过 15 年以上的发展，拥有庞大的开发者社区、丰富的库（cuDNN、cuBLAS、NCCL、cuFFT 等）和完善的工具链。SambaNova 虽然提供了 SambaFlow 编译器和 Pytorch 兼容层，但在**细粒度性能调试、复杂 custom operator 开发、大规模分布式训练**等方面与 CUDA 生态仍有差距。

8.3 设计哲学差异总结

表 4 总结了三类加速器在设计哲学上的核心差异。

表 4: NVIDIA GPU 与替代加速器的设计哲学对比

特性	**NVIDIA GPU**	**Groq LPU**	**SambaNova RDU**
执行模型	SIMT（动态调度）	确定性数据流	可重构数据流
核心资源	SM + Tensor Core	TSP + SRAM	RDU Tile + 多级存储
延迟-吞吐权衡	吞吐优先	延迟优先	两者兼顾（通过重构优化）
扩展方式	NVLink/NVSwitch 横向扩展	大量芯片串联	DataScale Memory 纵向扩展
软件生态	CUDA（极度成熟）	Groq Compiler	SambaFlow

总体而言，NVIDIA GPU 凭借其在通用性、可编程性和生态上的绝对优势，仍然是训练和绝大多数推理场景的首选。但 Groq 和 SambaNova 代表了两个重要的替代方向：**极致的低延迟确定性执行**，和**基于可重构数据流的大容量近存计算**。这些架构的兴衰，不仅取决于硬件设计的优劣，更取决于软件生态的完善程度和特定 AI 工作负载的演进方向。

9 研究趋势与未来挑战

9.1 面向 LLM 的架构瓶颈

当前以大语言模型（LLM）为代表的生成式 AI 对 GPU 架构提出了前所未有的挑战。相比传统的 CNN 训练，LLM 具有以下特点：巨大的模型参数量，千亿甚至万亿参数的模型需要海量的 HBM 容量；极长的上下文窗口，长序列注意力计算导致显存需求随序列长度平方增长（$O(N^2)$），并带来巨大的 KV Cache 存储压力；以及访存密集型特征，Transformer 的 Attention 层本质上是一个 memory-bound 操作，算力增长无法被有效利用。

这些瓶颈意味着，未来的 GPU 设计不仅要在 Tensor Core 的峰值算力上做文章，更要在**内存容量、内存带宽、多卡通信效率**上持续突破。

9.2 稀疏化、量化与近存计算

为了在现有硬件上运行越来越大的模型，学术界和工业界发展了多种优化方向。**稀疏化**方面，Ampere 的 2:4 结构化稀疏只是一个开始。未来的稀疏矩阵乘法可能会支持

更细粒度、更不规则的稀疏模式（如 block sparse、head sparse 等），硬件需要更灵活地跳过零值计算。**量化**方面，从 FP16 到 INT8 到 FP8 到 INT4/MXFP4，量化精度持续降低。硬件需要支持更灵活的混合精度数据通路，以及可配置的缩放因子（scaling factor）管理。**近存计算（Processing-in-Memory, PIM）**方面，将计算单元直接集成到 HBM 堆栈中，在存储器内部完成部分计算，减少数据搬运。虽然三星、SK 海力士等公司已推出基于 HBM-PIM 的原型产品，但如何将其与 GPU 现有的编程模型（如 CUDA）无缝集成，仍是一个开放问题。

9.3 领域专用 vs 通用可编程的权衡

Groq 和 SambaNova 的崛起，再次引发了关于" 领域专用加速器（DSA）vs 通用可编程 GPU" 的经典讨论。AI 工作负载的演进极快，从卷积网络到 Transformer 到 MoE 再到可能未来的新范式，DSA 的风险在于架构僵化。而 NVIDIA GPU 之所以能长期保持领先，正是因为其**可编程性**使得它能够灵活适应新的算法变化。

然而，随着 AI 推理逐渐成为云服务中的主要算力消耗，专用的低延迟推理芯片和基于数据流的重构加速器正在找到自己的市场细分。未来的高性能计算生态可能会呈现**多元共存**的格局：NVIDIA GPU 继续主导训练和通用推理，而 Groq、SambaNova、Cerebras 等则在特定场景中占据一席之地。

9.4 编译器-硬件协同设计的深化

一个贯穿本文的主题是，现代 GPU 的复杂性正越来越多地被" 卸载" 到编译器中。无论是 SIMD 到 SIMT 的演进，还是 Hanoi 微架构中编译器主导的重汇合管理，再到 compiler-controlled scoreboard 的引入，编译器-硬件协同设计（codesign）的重要性日益凸显。未来，随着架构异步性的增强、存储层次的复杂化、以及稀疏/量化指令的普及，**编译器优化将成为释放硬件性能的关键瓶颈**。围绕 GPU 编译器的研究（如 MLIR-based 中间表示、polyhedral model、自动 kernel 生成等）也将持续成为热点。

10 结论

本文围绕 NVIDIA GPU/GPGPU 处理器硬件架构进行了系统而深入的综述。从 GPGPU 的通用编程模型与执行模型出发，我们逐步深入到 SM 微架构、Tensor Core 的代际演进、内存层次与数据移动优化、多 GPU 互连机制，并最终将视角拓展到了替代加速器架构。

核心结论可以总结如下：

SIMT 是 GPGPU 执行模型的基石。它既保留了 MIMD 的编程灵活性，又通过 warp 级别的锁步执行获得了 SIMD 的高效率。CUDA 的线程层次结构和内存模型共同支撑了这一架构的高效运行。

SM 微架构的演进体现了高并行、高吞吐、深流水线的设计哲学。Warp Scheduler 从简单的" 单循环" 近似走向更复杂的双循环调度，并引入了 operand collector、scoreboard、

编译器控制位等机制，以支撑百级并发的指令级调度。SIMT Stack 的局限性最终催生了 Volta 之后基于显式同步的 Hanoi 微架构。

Tensor Core 是 NVIDIA GPU 在 AI 时代保持竞争力的核心技术。从 Volta 的第一代 warp-scoped MMA，到 Hopper 的 WGMMA 和 TMA，再到 Blackwell 的 TMEM 和第五代 Tensor Core，Tensor Core 的演进趋势是：协作粒度越来越大、异步化越来越强、精度越来越低、存储层次越来越复杂。

内存与数据移动是永恒的瓶颈。无论是单 SM 内部的 RF banking 与 operand collector，还是全局 HBM 的带宽限制，又或者是多 GPU 之间的 NVLink/NVSwitch 通信，数据搬运的成本始终困扰着高性能计算。异步数据搬运（cp.async、TMA、bulk tensor copy）和软件流水线技术是现代 kernel 优化的核心。

NVIDIA 之外，替代架构正在崛起。Groq LPU 的确定性执行和 SRAM 为中心设计，在 LLM 低延迟推理场景中展示了独特的优势；SambaNova RDU 的可重构数据流架构和大容量近存存储，为大模型推理提供了另一种技术路线。但它们能否挑战 NVIDIA 的霸主地位，仍取决于生态建设和工作负载演化的走向。

展望未来，随着 LLM 的模型规模和上下文窗口持续膨胀，GPU 架构需要在算力、内存容量、带宽和可扩展性之间进行更精细的权衡。领域专用加速器也将继续在特定市场中寻找生存空间。无论如何，对 GPGPU 微架构的深入理解，始终是从事高性能计算、体系结构和 AI 系统研究的必备基础。

致谢：本文部分内容在调研和写作过程中得到了 Claude (Anthropic) 的辅助，包括文献检索、内容整理、技术概念梳理和文本撰写。特此说明。

参考文献

[1] NVIDIA. NVIDIA Hopper Architecture In-Depth. 2022.

[2] NVIDIA. Tensor Core from Volta to Blackwell (Technical Analysis). 2024.

[3] 知乎专栏. GPGPU 编程模型与微架构入门. 2024.

[4] 知乎专栏. NVIDIA GPU 微架构设计与控制流优化：SIMT Stack 到 Hanoi 的演进. 2025.

[5] 知乎专栏. Analyzing Modern NVIDIA GPU cores. 2025.

[6] 知乎专栏. 深入解析 Modern GPU 的 control bits 设计. 2024.

[7] 知乎专栏. 带你从零开始设计 Warp Scheduler 硬件. 2026.

[8] ZxZhao et al. 带你从零开始 NV GPU 驱动开发 (Series). 2023–2025.

[9] NVIDIA Patent Analysis. NVIDIA Hopper TMA and cp.async/mbarrier Architectures. 2024.

[10] NVIDIA Patent Analysis. Hopper SWREMA Architecture Analysis. 2024.

[11] S. Li et al. GPU Interconnect and the Implications on Multi-GPU Performance. 2020.

[12] N. Muthukrishnan et al. Analyzing Modern NVIDIA GPU cores and Data Movement: More Memory Reference = More Energy. 2024.

[13] Huerta et al. Accel-Sim and HotChips NVIDIA GPU deep dive. 2024.

[14] Groq Technical Team. Groq LPU Architecture and Performance Analysis. 2024.

[15] SambaNova Systems. SambaNova SN40L RDU and DataFlow Architecture. 2024.

[16] J. Luo et al. Demystifying Tensor Cores to Optimize Half-Precision Matrix Multiply. 2020.

[17] 渣 B (zartbot). Inside Nvidia GPU: 谈谈 Blackwell 的不足并预测一下 Rubin 的微架构. 2025.